



Documentation technique · v1.0

Documentation API

MOTO Editor

Référence complète des moteurs IA, services, paramètres et agents intégrés. Architecture, flux de délégation et interfaces publiques.

Septembre 2026 · C# / .NET MAUI / WinUI 3

Sommaire

1. Vue d'ensemble de l'architecture

p.3

2. Moteurs IA internes (Moto.Core/AI/)

p.4

3. Services (Moto.Core/Services/)

p.6

4. Système de paramètres (data-driven)

p.7

5. Agents intégrés (AgentIntegrated/)

p.8

6. Flux de délégation Editor → AI → XENO

p.10

7. Références croisées et dépendances

p.11

8. Glossaire et conventions

p.12

Objectif du document

Cette documentation décrit l'ensemble des composants publics de MOTO Editor, leurs responsabilités, leurs interfaces et leurs interactions. Elle s'adresse aux développeurs contribuant au projet, aux intégrateurs souhaitant étendre les fonctionnalités, et aux utilisateurs avancés cherchant à comprendre les mécanismes sous-jacents.

Stack technique

C# 12

LANGUAGE

.NET 9

RUNTIME

MAUI

FRAMEWORK

WinUI 3

UI NATIVE

1. Vue d'ensemble de l'architecture

Principe de séparation des responsabilités

Doctrine fondamentale

- MOTO Editor repose sur une séparation stricte entre trois couches :
- **MOTO Editor** : édite, affiche, ouvre, sauvegarde, lance des commandes (jamais d'analyse profonde du code).
 - **MOTO AI** : assiste l'utilisateur, prédit, propose, route les demandes (jamais d'auto-application).
 - **XENO-SSS[∞]** : opérations structurées sur projet complet (analyse architecturale, génération, validation).

```
// Flux de délégation typique
Editor.UserAction → AI.RouteIntent() → XENO.ExecuteStructuredOperation()
                        ↑
                    Fallback: AI.LocalSuggestion()
```

Projets et responsabilités

Projet	Rôle	Exemples de composants
Moto.Editor	Shell UI MAUI/WinUI 3	MainPage.xaml, EditorPaneView, SettingsMenuView, TerminalView
Moto.Core	Logique métier et moteurs	MotoAiKernel, CortexEngine, TerminalService, SettingsEngine
Snake2000.Engine	Agents spécialisés	AgentScanner, AgentAnalyzer, AgentSynthesizer, XenoPipeline

Arborescence simplifiée

```
MOTO-Editor/
├─ Snake2000.Engine/AgentIntegrated/    ← 20 agents spécialisés
├─ Moto.Core/
│   ├── AI/                             ← Moteurs cognitifs
│   ├── Services/                       ← Build, Run, Terminal, Git
│   ├── Settings/                       ← Catalogue ~300 paramètres
│   └─ Performance/                    ← Modes Eco/Balanced/Turbo/Ultra
└─ Moto.Editor/                        ← Shell UI (Views, ViewModels, Pages)
```


2. Moteurs IA internes (Moto.Core/AI/)

Moteurs cognitifs principaux

Moteur	Rôle	Persistance	État
MotoAiKernel	Routeur d'intentions + fallback providers	—	Livré
ProjectUnderstandingEngine	Carte mentale (symboles, relations, issues)	—	Livré
CortexEngine	Mémoire cognitive + style utilisateur	.moto/cortex/memory.json	Livré
NeuralMode	Embeddings TF-IDF + retrieval sémantique	Index mémoire	Livré
AIWorkspace	Orchestrateur proactif (suggestions contextuelles)	—	Livré

Moteurs d'assistance

Moteur	Rôle	Persistance	État
AutoLinkEngine	Détection refs manquantes + Apply/Dismiss	—	Livré
ContextEngine	Suggestions senior proactives	—	Livré
TimeMachineEngine	Snapshots sans Git (historique granulaire)	.moto/timemachine/*.json	Livré
HealthMonitorEngine	Santé projet + complexité cyclomatique	—	Livré
PatternDetectorEngine	Détection ECS/Factory/Singleton/Observer	—	Livré

Moteurs de productivité

Moteur	Rôle	Persistance	État
DocEngine	Génération 6 markdown auto	.moto/docs/*.md	Livré
PlatformEngine	Portages multiplateforme + CI + validation incrémentale	—	Livré
PerformanceEngine	Modes Eco/Balanced/Turbo/Ultra	Settings	Livré

2. Moteurs IA internes (suite)

Moteurs pédagogiques et collaboratifs

Moteur	Rôle	Persistance	État
ExplainEverythingEngine	Explications pas-à-pas pour débutants	—	Livré
AiTutorEngine	Tutoriel interactif adaptatif	—	Livré
NoCodeOrchestrator	Génération visuelle sans code	—	Livré
PairProgrammingEngine	Programmation en binôme assistée	—	Livré
StoryModeEngine	Narration interactive du code	—	Livré
ProjectEvolutionEngine	Suivi de l'évolution architecturale	—	Livré

Interfaces publiques clés

MotoAiKernel

Point d'entrée unique pour toutes les requêtes IA. Route les intentions vers le moteur approprié avec fallback sur les providers locaux (Ollama) ou distants (si configurés).

```
public class MotoAiKernel {
    public async Task<AiResponse> RouteIntentAsync(IntentRequest request);
    public void RegisterProvider(IAiProvider provider, int priority);
    public AiResponse GetFallback(IntentRequest request);
}
```

CortexEngine

Maintient la mémoire cognitive de l'utilisateur : préférences de style, patterns récurrents, habitudes de navigation. Persiste dans `.moto/cortex/memory.json`.

```
public class CortexEngine {
    public void LearnFromAction(UserAction action);
    public StyleProfile GetCurrentStyle();
    public IEnumerable<Suggestion> GetContextualSuggestions(DocumentContext ctx);
}
```


3. Services (Moto.Core/Services/)

Services d'exécution

Service	Rôle	État
BuildEngine	Compilation incrémentale, détection d'erreurs, badges onglets	Livré
RunEngine	Lancement de projets, gestion des processus enfants	Livré
SandboxEngine	Exécution isolée avec verrou mot de passe	Livré
TerminalService	Terminal intégré avec historique, commandes, coloration	Livré

Services de gestion de projet

Service	Rôle	État
ProjectLockEngine	Verrouillage de fichiers pour édition collaborative	Livré
ProjectImportEngine	Import VS/VSCode, détection de configurations	Livré
FileTreeService	Explorateur de fichiers avec filtrage (bin, obj, node_modules)	Livré
ChatService	Gestion des threads de conversation IA	Livré
MotoAiService	Facade pour les appels IA depuis l'UI	Livré

Interfaces publiques clés

TerminalService

Gère le terminal intégré dans le dock du bas. Supporte l'historique des commandes, la coloration des erreurs, et l'ouverture automatique au lancement de projets.

```
public class TerminalService {
    public void ExecuteCommand(string command);
    public event EventHandler<TerminalOutput> OnOutput;
    public void Clear();
    public IEnumerable<string> GetHistory();
}
```


4. Système de paramètres (data-driven)

Architecture du catalogue

SettingsCatalog (classe partial)

Regroupe ~300 paramètres en 15 catégories. Chaque paramètre est défini de manière déclarative avec type, valeur par défaut, contrainte et catégorie.

```
public partial class SettingsCatalog {
    // Apparence
    public static readonly SettingDefinition<bool> ShowMinimap = new("editor.minimap.enabled", true);
    public static readonly SettingDefinition<int> FontSize = new("editor.fontSize", 14, min:8, max:24);
    public static readonly SettingDefinition<string> Theme = new("appearance.theme", "Moto Dark");

    // Éditeur
    public static readonly SettingDefinition<bool> WordWrap = new("editor.wordWrap", false);
    public static readonly SettingDefinition<int> TabSize = new("editor.tabSize", 4, min:1, max:8);

    // IA
    public static readonly SettingDefinition<bool> AiSuggestions = new("ai.suggestions.enabled", true);
    public static readonly SettingDefinition<string> AiProvider = new("ai.provider", "ollama");
}
```

Types de paramètres supportés

Type	Description	Exemple
Toggle	Booléen on/off	ShowMinimap
Int	Entier avec min/max	FontSize, TabSize
Enum	Liste de valeurs prédéfinies	Theme, PerformanceMode
String	Chaîne de caractères	AiProvider
Action	Bouton déclenchant une action	ResetToDefaults

Persistance et application

SettingsEngine.Shared

Singleton gérant la persistance dans %AppData%/MotoEditor/settings.json. Charge au démarrage, sauvegarde à chaque modification.

```
public class SettingsEngine {
    public static readonly SettingsEngine Shared = new();
}
```

```
public T Get<T>(SettingDefinition<T> definition);  
public void Set<T>(SettingDefinition<T> definition, T value);  
public void Save();  
public void Load();  
}
```

SettingsApplier

Applique les paramètres en temps réel : thème, police, minimap, indentation, etc. Écoute les changements via `INotifyPropertyChanged`.

5. Agents intégrés (AgentIntegrated/)

Pipeline XENO

Architecture des agents

Les agents spécialisés sont orchestrés par XenoPipeline et exécutent des tâches structurées sur le projet complet. Chaque agent a un rôle précis et retourne un AgentResult.

```
public class XenoPipeline {
    public async Task<AgentResult> ExecuteAsync(AgentContext context);
    public void RegisterAgent(IAgent agent, int priority);
}

public interface IAgent {
    string Name { get; }
    Task<AgentResult> RunAsync(AgentContext context);
    bool CanHandle(AgentContext context);
}
```

Agents principaux

Agent	Rôle	Priorité
AgentScanner	Scan complet du projet, détection de fichiers et dépendances	1
AgentAnalyzer	Analyse architecturale, détection de patterns et anti-patterns	2
AgentSynthesizer	Génération de fichiers, connexion des briques	3
AgentConnector	Connexion des dépendances, validation des imports	4
AgentValidator	Validation de cohérence, namespaces, types	5

Agents spécialisés

Agent	Rôle
ConsistencyScanner	Détection de fichiers/projets « fantômes », dépendances inutilisées
InconsistencyDetector	Détection d'incohérences structurelles
FileIntegrity	Vérification d'intégrité (hash, reconstruction)
StructuredLogCollector	Logs JSON structurés + explainabilité

DependencyResolver	Résolution automatique de dépendances manquantes
CodeRefactorer	Refactorisation structurée avec validation

5. Agents intégrés (suite)

Contexte et résultats

AgentContext

Objet passé à chaque agent contenant le contexte d'exécution : projet actif, fichiers concernés, requête utilisateur, historique.

```
public class AgentContext {  
    public Project ActiveProject { get; }  
    public IEnumerable<string> TargetFiles { get; }  
    public string UserQuery { get; }  
    public IDictionary<string, object> Metadata { get; }  
  
    public T GetMetadata<T>(string key, T defaultValue = default);  
    public void SetMetadata<T>(string key, T value);  
}
```

AgentResult

Résultat retourné par un agent. Peut contenir des fichiers modifiés, des suggestions, des erreurs, ou un état de succès/échec.

```
public class AgentResult {  
    public bool Success { get; }  
    public string Message { get; }  
    public IEnumerable<FileChange> Changes { get; }  
    public IEnumerable<Suggestion> Suggestions { get; }  
    public IEnumerable<Diagnostic> Diagnostics { get; }  
  
    public static AgentResult Succeeded(string message = null);  
    public static AgentResult Failed(string error);  
    public static AgentResult WithChanges(IEnumerable<FileChange> changes);  
}
```

Registre des agents

AgentRegistry

Registre central permettant d'enregistrer, découvrir et invoquer les agents. Supporte les priorités et le filtrage par contexte.

```
public class AgentRegistry {  
    public void Register(IAgent agent, int priority = 0);  
    public IEnumerable<IAgent> GetAgentsFor(AgentContext context);  
}
```

```
public IAgent GetByName(string name);  
}
```

6. Flux de délégation Editor → AI → XENO

Principe de délégation

Analogie avec les éditeurs existants

MOTO Editor délègue les opérations complexes à XENO-SSS[∞] de la même manière que :

- VS Code appelle Copilot
- Cursor appelle son backend IA
- Zed appelle Claude Code

Mais en **local**, **sans cloud**. XENO est l'orchestrateur structuré qui exécute les opérations sur le projet complet.

Flux typique : génération de projet

```
// 1. Utilisateur demande "Créer un projet MAUI avec MVVM"
Editor.UserAction("CreateProject", new { Template = "maui-mvvm" });

// 2. Editor route vers AI
var intent = await MotoAiKernel.RouteIntentAsync(new IntentRequest {
    Type = IntentType.ProjectGeneration,
    Query = "Créer un projet MAUI avec MVVM",
    Context = CurrentWorkspace
});

// 3. AI délègue à XENO
if (intent.RequiresStructuredOperation) {
    var xenoResult = await XenoBridge.ExecuteAsync(new XenoRequest {
        Operation = XenoOperation.CreateProject,
        Parameters = intent.Parameters,
        Agents = ["AgentScanner", "AgentSynthesizer", "AgentConnector", "AgentValidator"]
    });

    // 4. Editor applique les changements
    if (xenoResult.Success) {
        await FileTreeService.RefreshAsync();
        await BuildEngine.BuildAsync(xenoResult.GeneratedFiles);
    }
}
```

Flux typique : suggestion contextuelle

```
// 1. Utilisateur tape du code
Editor.OnTextChanged(document, position);

// 2. CortexEngine apprend de l'action
CortexEngine.LearnFromAction(new UserAction {
    Type = ActionType.CodeEdit,
    Document = document,
    Position = position,
    Timestamp = DateTime.Now
});

// 3. ContextEngine propose des suggestions
```

```
var suggestions = await ContextEngine.GetSuggestionsAsync(new DocumentContext {  
    Document = document,  
    CursorPosition = position,  
    RecentActions = CortexEngine.GetRecentActions(10)  
});  
  
// 4. UI affiche les suggestions (ghost text, panneau)  
Editor.ShowSuggestions(suggestions);
```

7. Références croisées et dépendances

Matrice de dépendances

Composant	Dépend de	Utilisé par
MotoAiKernel	Tous les moteurs IA	UI, Services, Agents
CortexEngine	—	ContextEngine, AIWorkspace
NeuralMode	EmbeddingEngine	CodeRetrieval, NeuralGenerator
TerminalService	—	TerminalView, RunEngine
SettingsEngine	—	UI, SettingsApplier, PerformanceEngine
XenoPipeline	AgentRegistry	MotoAiKernel, Editor

Conventions de nommage

Moteurs

- Suffixe Engine pour les composants exécutant une logique métier complexe.
- CortexEngine, NeuralMode, AutoLinkEngine

Services

- Suffixe Service pour les composants fournissant une capacité transversale.
- TerminalService, ChatService, FileTreeService

Agents

- Préfixe Agent pour les composants spécialisés exécutant des tâches structurées.
- AgentScanner, AgentAnalyzer, AgentSynthesizer

Persistance des données

Chemin	Contenu	Géré par
.moto/cortex/memory.json	Mémoire cognitive, style utilisateur	CortexEngine
.moto/timemachine/*.json	Snapshots granulaires	TimeMachineEngine

.moto/docs/*.md	Documentation auto-générée	DocEngine
%AppData%/MotoEditor/settings.json	Paramètres utilisateur	SettingsEngine

8. Glossaire et conventions

Termes clés

Terme	Définition
MOTO Editor	Éditeur de code ultra-léger, UI minimaliste, local-first
MOTO AI	Couche d'assistance IA locale (Ollama + moteurs cognitifs)
XENO-SSS [∞]	Orchestrateur IA structuré pour opérations sur projet complet
Cortex	Mémoire cognitive de l'utilisateur (style, habitudes)
Neural	Mode embeddings pour retrieval sémantique
AutoLink	Détection et résolution automatique de références manquantes
Time Machine	Snapshots granulaires sans Git
SettingsCatalog	Catalogue data-driven de ~300 paramètres

Conventions de code

Asynchronicité

Toutes les méthodes effectuant des opérations I/O ou IA sont asynchrones et retournent `Task<T>`.

Injection de dépendances

Les services sont injectés via le conteneur MAUI. Les moteurs sont des singletons accessibles via des propriétés statiques (ex: `SettingsEngine.Shared`).

Gestion des erreurs

Les erreurs sont capturées et exposées via des `Diagnostic` ou des `AgentResult.Failed`. L'UI affiche les erreurs dans les badges d'onglets et le panneau de diagnostics.

Contact et support

Pour toute question technique, contribution ou signalement de bug :

- GitHub Issues : github.com/moto-editor/moto-editor/issues
- Discussions : github.com/moto-editor/moto-editor/discussions

- Documentation utilisateur : voir *Manuel utilisateur*

© 2026 MOTO Editor · Documentation API v1.0 · Septembre 2026
Généré le 2 septembre 2026